TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based ETL pipeline. It defines data structures, transformation logic, runtime parameters, error handling, and operational behavior required to build a production-grade solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and generates analytical aggregations.

Scope

This TRD provides technical specifications for:

- AWS Glue job configuration and runtime parameters
- Source and target data schemas with column-level mappings
- Data ingestion from S3 CSV files using Spark DataFrames
- Data quality transformations (null removal, deduplication) with specific filter conditions
- AWS Glue Data Catalog table registration with DDL-equivalent metadata
- Apache Hudi table configuration for SCD Type 2 implementation
- Incremental processing logic using Hudi upsert operations
- Aggregation logic with Spark SQL transformations
- Error handling, logging, and validation rules
- File formats, partitioning strategies, and write modes

This TRD does NOT cover:

- Infrastructure provisioning (VPC, subnets, security groups)
- IAM role and policy definitions
- CloudWatch alarm configuration
- Cost optimization strategies
- Performance tuning beyond standard best practices
- Data retention and archival automation
- CI/CD pipeline configuration

Assumptions

1. AWS Glue version 4.0 or higher with Spark 3.3+ and Python 3.10+ runtime

2. Apache Hudi 0.12.0+ libraries are available in Glue environment via `--datalake-formats hudi`

3. CSV files use comma delimiter, double-quote text qualifier, and contain header row

4. CSV encoding is UTF-8

5. Date fields in Order CSV are in ISO 8601 format (YYYY-MM-DD) or parseable by Spark's default date parser

6. PricePerUnit and Qty are numeric types (decimal or integer)

7. CustId is a unique identifier in customer data and a foreign key in order data

8. OrderId is a unique identifier in order data

9. "Null" string matching is case-sensitive

10. Duplicate detection uses all columns with exact match (no fuzzy matching)

11. Glue database `gen_ai_poc_databrickscoe` exists prior to job execution

12. S3 bucket `adif-sdlc` exists with appropriate bucket policies

13. Glue job has IAM permissions for:

• s3:GetObject, s3:PutObject on s3://adif-sdlc/

• glue:GetDatabase, glue:CreateTable, glue:UpdateTable on database gen_ai_poc_databrickscoe

• logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents for CloudWatch

14. OpTs (operation timestamp) uses UTC timezone

15. StartDate and EndDate in SCD Type 2 use date type (not timestamp)

16. IsActive is a boolean field

17. Change detection for incremental processing relies on comparing current customer data against existing ordersummary records (no external change data capture mechanism)

18. Hudi table type is Copy-On-Write (COW) for ordersummary

19. Hudi record key for ordersummary is a composite of CustId and OrderId

20. Hudi precombine field for ordersummary is OpTs

21. No schema evolution is expected during initial implementation

22. Aggregation table (customeraggregatespend) uses overwrite mode on each run

23. Parquet format with Snappy compression for non-Hudi tables

24. No partitioning is specified for any table (can be added as optimization)

•

2. Technical Requirements

TR-INGEST-001: Ingest Customer Data from S3 CSV

• Related Functional Requirement ID(s): FR-INGEST-001

• Objective:

Read customer CSV files from S3 into a Spark DataFrame with schema inference and validation.

• Source Details:

• Source Type: S3 CSV Files

• Source Location: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Format: CSV with header

• Source Schema (Inferred):

• CustId: string (assumed, datatype not specified in FRD)

• Name: string (assumed, datatype not specified in FRD)

• EmailId: string (assumed, datatype not specified in FRD)

• Region: string (assumed, datatype not specified in FRD)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_customer_raw

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	string	N/A	Yes	
	Name	string	N/A	Yes	
	EmailId	string	N/A	Yes	
	Region	string	N/A	Yes	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Use Spark's `spark.read.format("csv")` with options:

• `header=true`

• `inferSchema=true`

• `mode=FAILFAST` (fail on malformed records)

2. Read all files matching pattern `s3://adif-sdlc/sdlc_wizard/customerdata/.csv`

3. Load into DataFrame `df_customer_raw`

4. Validate that required columns exist: CustId, Name, EmailId, Region

5. Log row count and schema

• Validation Rules:

• All four required columns must be present in CSV header

• At least one row must be present after header

• CSV must be parseable without format exceptions

• Error Handling and Logging:

• If S3 path does not exist or is empty: raise `FileNotFoundException`, log error, halt job with exit code 1

• If CSV is malformed: raise `ParseException`, log error with file name, halt job with exit code 1

• If required columns are missing: raise `AnalysisException`, log error with missing column names, halt job with exit code 1

• Log INFO: "Customer data ingestion started from s3://adif-sdlc/sdlc_wizard/customerdata/"

• Log INFO: "Customer data ingestion completed. Row count: {count}"

• Runtime Parameters:

• `--SOURCE_CUSTOMER_PATH`: s3://adif-sdlc/sdlc_wizard/customerdata/

• `--CSV_DELIMITER`: ","

• `--CSV_QUOTE`: "\""

• `--CSV_HEADER`: "true"

• Operational Notes:

• Use Glue DynamicFrame if schema evolution is needed in future; otherwise Spark DataFrame is sufficient

• Consider enabling Glue job bookmark if incremental file processing is required (not specified in FRD)

•

TR-INGEST-002: Ingest Order Data from S3 CSV

• Related Functional Requirement ID(s): FR-INGEST-002

• Objective:

Read order CSV files from S3 into a Spark DataFrame with schema inference and validation.

• Source Details:

• Source Type: S3 CSV Files

• Source Location: s3://adif-sdlc/sdlc_wizard/orderdata/

• Source Format: CSV with header

• Source Schema (Inferred):

• OrderId: string (assumed, datatype not specified in FRD)

• ItemName: string (assumed, datatype not specified in FRD)

• PricePerUnit: decimal (assumed, datatype not specified in FRD)

• Qty: integer (assumed, datatype not specified in FRD)

• Date: date (assumed, datatype not specified in FRD)

• CustId: string (assumed, datatype not specified in FRD)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_order_raw

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	string	N/A	Yes	
	ItemName	string	N/A	Yes	
	PricePerUnit	decimal	10,2 (assumed)	Yes	
	Qty	integer	N/A	Yes	
	Date	date	N/A	Yes	
	CustId	string	N/A	Yes	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Use Spark's `spark.read.format("csv")` with options:

• `header=true`

• `inferSchema=true`

• `mode=FAILFAST`

2. Read all files matching pattern `s3://adif-sdlc/sdlc_wizard/orderdata/.csv`

3. Load into DataFrame `df_order_raw`

4. Validate that required columns exist: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

5. Cast Date column to date type if inferred as string

6. Cast PricePerUnit to decimal(10,2) if inferred differently

7. Cast Qty to integer if inferred differently

8. Log row count and schema

• Validation Rules:

• All six required columns must be present in CSV header

• At least one row must be present after header

• CSV must be parseable without format exceptions

• Date values must be convertible to date type

• Error Handling and Logging:

• If S3 path does not exist or is empty: raise `FileNotFoundException`, log error, halt job with exit code 1

• If CSV is malformed: raise `ParseException`, log error with file name, halt job with exit code 1

• If required columns are missing: raise `AnalysisException`, log error with missing column names, halt job with exit code 1

• If date parsing fails: raise `DateTimeException`, log error with problematic value, halt job with exit code 1

• Log INFO: "Order data ingestion started from s3://adif-sdlc/sdlc_wizard/orderdata/"

• Log INFO: "Order data ingestion completed. Row count: {count}"

• Runtime Parameters:

• `--SOURCE_ORDER_PATH`: s3://adif-sdlc/sdlc_wizard/orderdata/

• `--CSV_DELIMITER`: ","

• `--CSV_QUOTE`: "\""

• `--CSV_HEADER`: "true"

• Operational Notes:

• Ensure numeric type casting does not cause precision loss

• Consider adding data quality checks for negative PricePerUnit or Qty values (not specified in FRD)

•

TR-CLEAN-001: Remove Null Values from Customer Data

• Related Functional Requirement ID(s): FR-CLEAN-001

• Objective:

Filter out customer records containing the string "Null" or actual NULL values in any column.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_customer_raw (from TR-INGEST-001)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_customer_null_removed

• Input Schema:

Same as TR-INGEST-001 Output Schema

• Output Schema:

Same as Input Schema (filtered subset)

• Processing / Transformation Logic:

1. Create filter condition for each column:

• `(col("CustId") != "Null") & (col("CustId").isNotNull())`

• `(col("Name") != "Null") & (col("Name").isNotNull())`

• `(col("EmailId") != "Null") & (col("EmailId").isNotNull())`

• `(col("Region") != "Null") & (col("Region").isNotNull())`

2. Combine all conditions with AND operator

3. Apply filter to df_customer_raw

4. Store result in df_customer_null_removed

5. Log row count before and after filtering

• Validation Rules:

• String comparison for "Null" is case-sensitive

• Both string "Null" and actual NULL must be removed

• Error Handling and Logging:

• If all rows are removed: log WARNING: "All customer records removed due to null values. Row count: 0"

• Log INFO: "Customer null removal started. Initial row count: {before_count}"

• Log INFO: "Customer null removal completed. Final row count: {after_count}. Removed: {removed_count}"

• Runtime Parameters:

• `--NULL_STRING_VALUE`: "Null"

• Operational Notes:

• Consider making null string value configurable for different data sources

• Performance: filter operation is lazy; actual execution occurs on action

•

TR-CLEAN-002: Remove Null Values from Order Data

• Related Functional Requirement ID(s): FR-CLEAN-002

• Objective:

Filter out order records containing the string "Null" or actual NULL values in any column.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_order_raw (from TR-INGEST-002)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_order_null_removed

• Input Schema:

Same as TR-INGEST-002 Output Schema

• Output Schema:

Same as Input Schema (filtered subset)

• Processing / Transformation Logic:

1. Create filter condition for each column:

• `(col("OrderId") != "Null") & (col("OrderId").isNotNull())`

• `(col("ItemName") != "Null") & (col("ItemName").isNotNull())`

• `(col("PricePerUnit").isNotNull())`

• `(col("Qty").isNotNull())`

• `(col("Date").isNotNull())`

• `(col("CustId") != "Null") & (col("CustId").isNotNull())`

2. Combine all conditions with AND operator

3. Apply filter to df_order_raw

4. Store result in df_order_null_removed

5. Log row count before and after filtering

• Validation Rules:

• String comparison for "Null" is case-sensitive for string columns

• Numeric and date columns only check for actual NULL (not string "Null")

• Error Handling and Logging:

• If all rows are removed: log WARNING: "All order records removed due to null values. Row count: 0"

• Log INFO: "Order null removal started. Initial row count: {before_count}"

• Log INFO: "Order null removal completed. Final row count: {after_count}. Removed: {removed_count}"

• Runtime Parameters:

• `--NULL_STRING_VALUE`: "Null"

• Operational Notes:

• Numeric columns (PricePerUnit, Qty) and date columns (Date) cannot contain string "Null" after type casting, so only NULL check is needed

•

TR-CLEAN-003: Remove Duplicate Customer Records

• Related Functional Requirement ID(s): FR-CLEAN-003

• Objective:

Deduplicate customer records based on exact match across all columns.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_customer_null_removed (from TR-CLEAN-001)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_customer_clean

• Input Schema:

Same as TR-CLEAN-001 Output Schema

• Output Schema:

Same as Input Schema (deduplicated subset)

• Processing / Transformation Logic:

1. Apply Spark `dropDuplicates()` method without specifying columns (uses all columns)

2. Store result in df_customer_clean

3. Log row count before and after deduplication

• Validation Rules:

• Deduplication considers all columns: CustId, Name, EmailId, Region

• Exact match required (no fuzzy matching or normalization)

• Error Handling and Logging:

• Log INFO: "Customer deduplication started. Initial row count: {before_count}"

• Log INFO: "Customer deduplication completed. Final row count: {after_count}. Duplicates removed: {removed_count}"

• Runtime Parameters:

• None

• Operational Notes:

• Spark's dropDuplicates() is non-deterministic in which duplicate to keep; consider adding orderBy if deterministic selection is needed

• Performance: deduplication triggers shuffle operation

•

TR-CLEAN-004: Remove Duplicate Order Records

• Related Functional Requirement ID(s): FR-CLEAN-004

• Objective:

Deduplicate order records based on exact match across all columns.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_order_null_removed (from TR-CLEAN-002)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_order_clean

• Input Schema:

Same as TR-CLEAN-002 Output Schema

• Output Schema:

Same as Input Schema (deduplicated subset)

• Processing / Transformation Logic:

1. Apply Spark `dropDuplicates()` method without specifying columns (uses all columns)

2. Store result in df_order_clean

3. Log row count before and after deduplication

• Validation Rules:

• Deduplication considers all columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Exact match required (no fuzzy matching or normalization)

• Error Handling and Logging:

• Log INFO: "Order deduplication started. Initial row count: {before_count}"

• Log INFO: "Order deduplication completed. Final row count: {after_count}. Duplicates removed: {removed_count}"

• Runtime Parameters:

• None

• Operational Notes:

• Consider adding orderBy on OrderId if deterministic duplicate selection is needed

• Performance: deduplication triggers shuffle operation

•

TR-CATALOG-001: Register Customer Table in Glue Data Catalog

• Related Functional Requirement ID(s): FR-CATALOG-001

• Objective:

Write cleaned customer data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_customer_clean (from TR-CLEAN-003)

• Target Details:

• Target Type: S3 Parquet Files + Glue Catalog Table

• Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/customer/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: sdlc_wizard_customer

• Target Format: Parquet with Snappy compression

• Input Schema:

Same as TR-CLEAN-003 Output Schema

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	string	N/A	No	
	Name	string	N/A	No	

	EmailId	string	N/A	No	
	Region	string	N/A	No	

• Processing / Transformation Logic:

1. Write df_customer_clean to S3 using:

• `df_customer_clean.write.format("parquet")`

• `mode("overwrite")`

• `option("compression", "snappy")`

• `save("s3://adif-sdlc/curated/sdlc_wizard/customer/")`

2. Register table in Glue Data Catalog using Spark SQL:

```sql
CREATE EXTERNAL TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard_customer (
CustId STRING,
Name STRING,
EmailId STRING,
Region STRING
)
STORED AS PARQUET
LOCATION 's3://adif-sdlc/curated/sdlc_wizard/customer/'
```

3. Run `MSCK REPAIR TABLE` if partitions are added in future

4. Log table registration success

• Validation Rules:

• Database gen_ai_poc_databrickscoe must exist

• S3 path must be writable

• All columns must be non-null after cleaning stages

• Error Handling and Logging:

• If database does not exist: raise `AnalysisException`, log error, halt job with exit code 1

• If S3 write fails: raise `IOException`, log error with S3 path, halt job with exit code 1

• If table registration fails: raise `AnalysisException`, log error, halt job with exit code 1

• Log INFO: "Writing customer data to s3://adif-sdlc/curated/sdlc_wizard/customer/"

• Log INFO: "Customer table registered in Glue Catalog: gen_ai_poc_databrickscoe.sdlc_wizard_customer"

• Runtime Parameters:

• `--TARGET_CUSTOMER_PATH`: s3://adif-sdlc/curated/sdlc_wizard/customer/

• `--GLUE_DATABASE`: gen_ai_poc_databrickscoe

• `--CUSTOMER_TABLE_NAME`: sdlc_wizard_customer

• Operational Notes:

• Overwrite mode replaces all data on each run; consider partitioning by Region for incremental updates

• Glue Catalog table can be queried via Athena immediately after registration

•

TR-CATALOG-002: Register Order Table in Glue Data Catalog

• Related Functional Requirement ID(s): FR-CATALOG-002

• Objective:

Write cleaned order data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_order_clean (from TR-CLEAN-004)

• Target Details:

• Target Type: S3 Parquet Files + Glue Catalog Table

• Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/order/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: sdlc_wizard_order

• Target Format: Parquet with Snappy compression

• Input Schema:

Same as TR-CLEAN-004 Output Schema

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	string	N/A	No	
	ItemName	string	N/A	No	
	PricePerUnit	decimal	10,2	No	
	Qty	integer	N/A	No	
	Date	date	N/A	No	
	CustId	string	N/A	No	

• Processing / Transformation Logic:

1. Write df_order_clean to S3 using:

• `df_order_clean.write.format("parquet")`

• `mode("overwrite")`

• `option("compression", "snappy")`

• `save("s3://adif-sdlc/curated/sdlc_wizard/order/")`

2. Register table in Glue Data Catalog using Spark SQL:

```sql
CREATE EXTERNAL TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard_order (
OrderId STRING,
ItemName STRING,
PricePerUnit DECIMAL(10,2),
Qty INT,
Date DATE,
CustId STRING
)
STORED AS PARQUET
LOCATION 's3://adif-sdlc/curated/sdlc_wizard/order/'
```

3. Run `MSCK REPAIR TABLE` if partitions are added in future

4. Log table registration success

• Validation Rules:

• Database gen_ai_poc_databrickscoe must exist

• S3 path must be writable

• All columns must be non-null after cleaning stages

• PricePerUnit and Qty must be numeric

• Error Handling and Logging:

• If database does not exist: raise `AnalysisException`, log error, halt job with exit code 1

• If S3 write fails: raise `IOException`, log error with S3 path, halt job with exit code 1

• If table registration fails: raise `AnalysisException`, log error, halt job with exit code 1

• Log INFO: "Writing order data to s3://adif-sdlc/curated/sdlc_wizard/order/"

• Log INFO: "Order table registered in Glue Catalog: gen_ai_poc_databrickscoe.sdlc_wizard_order"

• Runtime Parameters:

• `--TARGET_ORDER_PATH`: s3://adif-sdlc/curated/sdlc_wizard/order/

• `--GLUE_DATABASE`: gen_ai_poc_databrickscoe

• `--ORDER_TABLE_NAME`: sdlc_wizard_order

• Operational Notes:

• Overwrite mode replaces all data on each run; consider partitioning by Date for incremental updates

• Glue Catalog table can be queried via Athena immediately after registration

•

TR-SCD2-001: Implement SCD Type 2 for Order Summary Using Apache Hudi

• Related Functional Requirement ID(s): FR-SCD2-001

• Objective:

Join customer and order data, then write to ordersummary table using Apache Hudi with SCD Type 2 logic to track historical changes in customer attributes.

• Source Details:

• Source Type 1: In-memory Spark DataFrame

• Source Name 1: df_customer_clean (from TR-CLEAN-003)

• Source Type 2: In-memory Spark DataFrame

• Source Name 2: df_order_clean (from TR-CLEAN-004)

• Source Type 3: Existing Hudi Table (if present)

• Source Name 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Details:

• Target Type: S3 Hudi Table + Glue Catalog Table

• Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: ordersummary

• Target Format: Apache Hudi (Copy-On-Write)

• Input Schema (df_customer_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	string	N/A	No	
	Name	string	N/A	No	
	EmailId	string	N/A	No	

	Region	string	N/A	No	

• Input Schema (df_order_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	string	N/A	No	
	ItemName	string	N/A	No	
	PricePerUnit	decimal	10,2	No	
	Qty	integer	N/A	No	
	Date	date	N/A	No	
	CustId	string	N/A	No	

• Output Schema (ordersummary):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	---	---	---	---	---	
	CustId	string	N/A	No	Direct map from df_customer_clean.CustId	
	Name	string	N/A	No	Direct map from df_customer_clean.Name	
	EmailId	string	N/A	No	Direct map from df_customer_clean.EmailId	
	Region	string	N/A	No	Direct map from df_customer_clean.Region	
	OrderId	string	N/A	No	Direct map from df_order_clean.OrderId	
	ItemName	string	N/A	No	Direct map from df_order_clean.ItemName	
	PricePerUnit	decimal	10,2	No	Direct map from df_order_clean.PricePerUnit	
	Qty	integer	N/A	No	Direct map from df_order_clean.Qty	
	Date	date	N/A	No	Direct map from df_order_clean.Date	
	IsActive	boolean	N/A	No	Derived: true for new/updated records	
	StartDate	date	N/A	No	Derived: current_date() for new/updated	
	EndDate	date	N/A	Yes	Derived: null for active, current_date() for closed	
	OpTs	timestamp	N/A	No	Derived: current_timestamp() in UTC	

• Processing / Transformation Logic:

1. Join Customer and Order Data:

```python
df_joined = df_customer_clean.join(df_order_clean, "CustId", "inner")
```

2. Add SCD Type 2 Metadata Columns:

```python
from pyspark.sql.functions import current_date, current_timestamp, lit
```

```
df_joined = df_joined.withColumn("IsActive", lit(True)) \
.withColumn("StartDate", current_date()) \
.withColumn("EndDate", lit(None).cast("date")) \
.withColumn("OpTs", current_timestamp())
```

3. Read Existing Hudi Table (if exists):

```python
if hudi_table_exists("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/"):
df_existing = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
else:
df_existing = None
```

4. Detect Changes (if existing table present):

- For each record in df_joined, check if matching active record exists in df_existing
- Match criteria: CustId AND OrderId
- Change detection: compare Name, EmailId, Region
- If change detected:
- Mark existing record as inactive: IsActive=false, EndDate=current_date(), OpTs=current_timestamp()
- Insert new record with updated values: IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()
- If no change detected: skip (no action)
- If no matching record exists: insert as new active record

5. Prepare Hudi Write Configuration:

```python
hudi_options = {
'hoodie.table.name': 'ordersummary',
'hoodie.datasource.write.recordkey.field': 'CustId,OrderId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.partitionpath.field': '',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.hive_style_partitioning': 'false',
'hoodie.upsert.shuffle.parallelism': 2,
```

'hoodie.insert.shuffle.parallelism': 2

}

```

6. Write to Hudi Table:

```python

df_to_write.write.format("hudi") \

.options(hudi_options) \

.mode("append") \

.save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")

```

7. Register/Update Glue Catalog Table:

```python

spark.sql(f"""

CREATE EXTERNAL TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.ordersummary (

CustId STRING,

Name STRING,

EmailId STRING,

Region STRING,

OrderId STRING,

ItemName STRING,

PricePerUnit DECIMAL(10,2),

Qty INT,

Date DATE,

IsActive BOOLEAN,

StartDate DATE,

EndDate DATE,

OpTs TIMESTAMP

)

STORED AS PARQUET

LOCATION 's3://adif-sdlc/curated/sdlc_wizard/ordersummary/'

""")

```

- Validation Rules:
- Join must produce at least one record (inner join may result in zero records if no CustId match)
```

• IsActive must be boolean (true/false)

• StartDate must be <= current_date()

• EndDate must be null for active records, non-null for inactive records

• OpTs must be in UTC timezone

• Hudi record key (CustId, OrderId) must be unique per active record

• Error Handling and Logging:

• If join produces zero records: log WARNING: "Customer-Order join produced no results. No ordersummary records to write."

• If Hudi write fails: raise `HoodieException`, log error with S3 path, halt job with exit code 1

• If Glue Catalog registration fails: log ERROR but do not halt (table can be manually registered)

• Log INFO: "SCD Type 2 processing started for ordersummary"

• Log INFO: "Customer-Order join completed. Joined row count: {count}"

• Log INFO: "Hudi upsert completed. Records written: {count}"

• Log INFO: "Ordersummary table registered/updated in Glue Catalog"

• Runtime Parameters:

• `--TARGET_ORDERSUMMARY_PATH`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• `--GLUE_DATABASE`: gen_ai_poc_databrickscoe

• `--ORDERSUMMARY_TABLE_NAME`: ordersummary

• `--HUDI_TABLE_TYPE`: COPY_ON_WRITE

• `--HUDI_RECORD_KEY`: CustId,OrderId

• `--HUDI_PRECOMBINE_KEY`: OpTs

• Operational Notes:

• First run will create Hudi table; subsequent runs will upsert

• Hudi's upsert operation handles SCD Type 2 logic by comparing precombine field (OpTs)

• For true SCD Type 2 with explicit close-and-insert logic, custom merge logic may be required beyond standard Hudi upsert

• Consider implementing custom SCD Type 2 logic using Hudi's merge-on-read or manual DataFrame operations if standard upsert does not meet requirements

• Hudi table can be queried via Athena using Hudi-specific query syntax

•

TR-INCR-001: Trigger Incremental SCD2 Updates on Customer Changes

• Related Functional Requirement ID(s): FR-INCR-001

• Objective:

Detect changes in customer data and trigger incremental updates to ordersummary table using SCD Type 2 logic, processing only changed customer records.

- Source Details:
- Source Type 1: In-memory Spark DataFrame
- Source Name 1: df_customer_clean (from TR-CLEAN-003)
- Source Type 2: Existing Hudi Table
- Source Name 2: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Target Details:
- Target Type: S3 Hudi Table (incremental update)
- Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Database: gen_ai_poc_databrickscoe
- Target Table: ordersummary

- Input Schema:

Same as TR-SCD2-001 Input Schemas

- Output Schema:

Same as TR-SCD2-001 Output Schema

- Processing / Transformation Logic:

1. Read Existing Ordersummary Table:

```python
df_existing_ordersummary = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

2. Filter Active Records:

```python
df_active_ordersummary = df_existing_ordersummary.filter(col("IsActive") == True)
```

3. Identify Changed Customers:

```python
# Get distinct customers from existing active ordersummary
df_existing_customers = df_active_ordersummary.select("CustId", "Name", "EmailId", "Region").distinct()

# Left anti join to find new customers
```

df_new_customers = df_customer_clean.join(df_existing_customers, "CustId", "left_anti")

Join to find changed customers (same CustId but different attributes)

df_changed_customers = df_customer_clean.join(

df_existing_customers,

(df_customer_clean.CustId == df_existing_customers.CustId) &

((df_customer_clean.Name != df_existing_customers.Name)	
(df_customer_clean.EmailId != df_existing_customers.EmailId)	

(df_customer_clean.Region != df_existing_customers.Region)),

"inner"

).select(df_customer_clean[""])

Union new and changed customers

df_customers_to_process = df_new_customers.union(df_changed_customers)

```

4. Retrieve Orders for Changed Customers:

```python

df_orders_to_process = df_order_clean.join(df_customers_to_process, "CustId", "inner")

```

5. Apply SCD Type 2 Logic (same as TR-SCD2-001):

• For changed customers: close existing active records, insert new active records

• For new customers: insert new active records

• Use Hudi upsert operation

6. Write Incremental Updates:

```python

# Same Hudi write configuration as TR-SCD2-001

df_to_write.write.format("hudi") \

.options(hudi_options) \

.mode("append") \

.save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")

```

• Validation Rules:

• Change detection must compare all customer attributes: Name, EmailId, Region

• Only customers with actual changes should trigger updates
```

• Unchanged customers should not generate new ordersummary records

• Error Handling and Logging:

• If no changes detected: log INFO: "No customer changes detected. Skipping incremental processing." and exit gracefully

• If change detection fails: raise `AnalysisException`, log error, halt job with exit code 1

• If Hudi upsert fails: raise `HoodieException`, log error, halt job with exit code 1

• Log INFO: "Incremental processing started. Checking for customer changes."

• Log INFO: "Customer changes detected. New customers: {new_count}, Changed customers: {changed_count}"

• Log INFO: "Incremental SCD Type 2 update completed. Records processed: {count}"

• Runtime Parameters:

• Same as TR-SCD2-001

• Operational Notes:

• Incremental processing assumes ordersummary table already exists (initial load via TR-SCD2-001 required)

• Change detection relies on comparing current customer data against active ordersummary records

• Consider using Glue job bookmarks or external change tracking mechanism for more efficient change detection

• Performance: incremental processing reduces compute time by processing only changed data

•

TR-AGG-001: Generate Customer Aggregate Spend Table

• Related Functional Requirement ID(s): FR-AGG-001

• Objective:

Aggregate order data by customer and date to calculate total spending, then write results to analytics table.

• Source Details:

• Source Type 1: In-memory Spark DataFrame

• Source Name 1: df_customer_clean (from TR-CLEAN-003)

• Source Type 2: In-memory Spark DataFrame

• Source Name 2: df_order_clean (from TR-CLEAN-004)

• Target Details:

• Target Type: S3 Parquet Files + Glue Catalog Table

• Target S3 Location: s3://adif-sdlc/analytics/customeraggregatespend/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: customeraggregatespend

• Target Format: Parquet with Snappy compression

• Input Schema (df_customer_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	string	N/A	No	
	Name	string	N/A	No	
	EmailId	string	N/A	No	
	Region	string	N/A	No	

• Input Schema (df_order_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	string	N/A	No	
	ItemName	string	N/A	No	
	PricePerUnit	decimal	10,2	No	
	Qty	integer	N/A	No	
	Date	date	N/A	No	
	CustId	string	N/A	No	

• Output Schema (customeraggregatespend):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	---	---	---	---	---	
	Name	string	N/A	No	Direct map from df_customer_clean.Name	
	TotalAmount	decimal	20,2	No	Derived: SUM(df_order_clean.PricePerUnit × df_order_clean.Qty)	
	Date	date	N/A	No	Direct map from df_order_clean.Date	

• Processing / Transformation Logic:

1. Join Customer and Order Data:

```python
df_joined = df_customer_clean.join(df_order_clean, "CustId", "inner")
```

2. Calculate TotalAmount per Order:

```python
from pyspark.sql.functions import col
```

df_with_total = df_joined.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))
```

3. Aggregate by Customer Name and Date:

```python
from pyspark.sql.functions import sum as _sum

df_aggregated = df_with_total.groupBy("Name", "Date") \
.agg(_sum("TotalAmount").alias("TotalAmount"))
```

4. Select Final Columns:

```python
df_final = df_aggregated.select("Name", "TotalAmount", "Date")
```

5. Write to S3:

```python
df_final.write.format("parquet") \
.mode("overwrite") \
.option("compression", "snappy") \
.save("s3://adif-sdlc/analytics/customeraggregatespend/")
```

6. Register Glue Catalog Table:

```python
spark.sql(f"""
CREATE EXTERNAL TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.customeraggregatespend (
Name STRING,
TotalAmount DECIMAL(20,2),
Date DATE
)
STORED AS PARQUET
LOCATION 's3://adif-sdlc/analytics/customeraggregatespend/'
""")
```

• Validation Rules:
```

• PricePerUnit and Qty must be numeric (validated in TR-CLEAN-002)

• TotalAmount must be non-negative (business rule: negative amounts indicate data quality issue)

• Each customer-date combination should have exactly one row in output

• Error Handling and Logging:

• If join produces zero records: log WARNING: "Customer-Order join produced no results. No aggregate spend to calculate."

• If PricePerUnit or Qty are non-numeric after cleaning: log ERROR and exclude those orders from aggregation

• If aggregation produces negative TotalAmount: log WARNING with customer name and date

• If S3 write fails: raise `IOException`, log error with S3 path, halt job with exit code 1

• If Glue Catalog registration fails: log ERROR but do not halt

• Log INFO: "Customer aggregate spend calculation started"

• Log INFO: "Aggregation completed. Unique customer-date combinations: {count}"

• Log INFO: "Customer aggregate spend table written to s3://adif-sdlc/analytics/customeraggregatespend/"

• Log INFO: "Customer aggregate spend table registered in Glue Catalog"

• Runtime Parameters:

• `--TARGET_AGGREGATE_PATH`: s3://adif-sdlc/analytics/customeraggregatespend/

• `--GLUE_DATABASE`: gen_ai_poc_databrickscoe

• `--AGGREGATE_TABLE_NAME`: customeraggregatespend

• Operational Notes:

• Overwrite mode replaces all data on each run (full refresh)

• Consider partitioning by Date for better query performance

• TotalAmount precision (20,2) allows for large aggregate values

• Customers with no orders will not appear in aggregate table

•

3. Runtime Strategy Notes

Authoring Language

• Primary Language: Python 3.10+

• Framework: PySpark 3.3+ (AWS Glue 4.0 runtime)

• Libraries:

• Apache Hudi 0.12.0+ (via `--datalake-formats hudi` Glue job parameter)

• AWS Glue Python libraries (awsglue.context, awsglue.job, awsglue.dynamicframe)

• PySpark SQL functions (pyspark.sql.functions)

Execution Strategy

• Job Type: AWS Glue ETL Job (Spark)

• Glue Version: 4.0

• Worker Type: G.1X (recommended for standard workloads) or G.2X (for larger datasets)

• Number of Workers: 2-10 (scale based on data volume)

• Job Timeout: 2880 minutes (48 hours max)

• Max Retries: 0 (manual retry recommended for debugging)

• Execution Mode: Batch processing (scheduled via EventBridge or Glue Workflows)

• Concurrency: Single job execution (no parallel runs to avoid SCD Type 2 conflicts)

Glue Job Type Expectations

• Job Type: Spark ETL job (not Python Shell)

• Script Location: s3:///scripts/customer_order_etl.py

• Temporary Directory: s3:///temp/

• Spark UI Logs: Enabled (for debugging)

• Continuous Logging: Enabled (CloudWatch Logs)

• Job Bookmark: Disabled (full refresh mode; enable for incremental file processing if needed)

• Security Configuration: Use Glue security configuration with encryption at rest and in transit

• IAM Role: Glue service role with policies for S3, Glue Catalog, CloudWatch Logs

•

4. Data Design Details

Source Paths / Source Systems

	Source Name	Source Type	Source Path	Format	Schema Defined In	
	---	---	---	---	---	
	Customer CSV	S3 File	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	TR-INGEST-001	
	Order CSV	S3 File	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	TR-INGEST-002	
	Existing Ordersummary	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi	TR-SCD2-001	

Target Paths / Target Tables

	Target Type	Target Path	Format	Write Mode	Catalog Database	Catalo
---	---	---	---	---	---	---
	S3 Parquet	s3://adif-sdlc/curated/sdlc_wizard/customer/	Parquet	Overwrite	gen_ai_poc_databrickscoe	sdlc_w
	S3 Parquet	s3://adif-sdlc/curated/sdlc_wizard/order/	Parquet	Overwrite	gen_ai_poc_databrickscoe	sdlc_w

CD2	S3 Hudi	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi	Append/Upsert	gen_ai_poc_databrickscoe	orders
ate Spend	S3 Parquet	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	Overwrite	gen_ai_poc_databrickscoe	custom

Catalog Registration Expectations

- All tables must be registered in Glue Data Catalog database: `gen_ai_poc_databrickscoe`
- Table registration occurs immediately after data write
- Use `CREATE EXTERNAL TABLE IF NOT EXISTS` to avoid errors on re-runs
- Hudi tables require Hudi-specific Glue Catalog configuration (handled by Hudi write operation)
- Parquet tables use standard Glue Catalog DDL

Partitioning Expectations

- Current Implementation: No partitioning (as per FRD)
- Future Optimization Recommendations:
- Customer table: partition by Region
- Order table: partition by Date (year/month/day)
- Ordersummary table: partition by StartDate (year/month)
- Customer Aggregate Spend: partition by Date (year/month)

File Formats / Table Formats

	Table Name	Storage Format	Compression	Table Type	Notes	
	---	---	---	---	---	
	sdlc_wizard_customer	Parquet	Snappy	External	Columnar format for efficient querying	
	sdlc_wizard_order	Parquet	Snappy	External	Columnar format for efficient querying	
	ordersummary	Hudi (Parquet)	Snappy	External	Copy-On-Write table type for SCD Type 2	
	customeraggregatespend	Parquet	Snappy	External	Columnar format for analytical queries	

Update / Overwrite / Append Behavior

	Table Name	Write Mode	Behavior	
	---	---	---	
	sdlc_wizard_customer	Overwrite	Full refresh on each run; all existing data replaced	
	sdlc_wizard_order	Overwrite	Full refresh on each run; all existing data replaced	
	ordersummary	Append/Upsert	Hudi upsert: inserts new records, updates existing based on record key	
	customeraggregatespend	Overwrite	Full refresh on each run; all existing data replaced	

Data Lineage Notes

- Customer CSV → df_customer_raw → df_customer_null_removed → df_customer_clean → sdlc_wizard_customer (Glue Catalog)
- Order CSV → df_order_raw → df_order_null_removed → df_order_clean → sdlc_wizard_order (Glue Catalog)

• df_customer_clean + df_order_clean → df_joined → ordersummary (Hudi table with SCD Type 2)

• df_customer_clean + df_order_clean → df_aggregated → customeraggregatespend (Parquet table)

•

5. Processing Details

Data Quality Rules

e ID	Rule Description	Applied To	Implementation
-----	----------	----------	----------
-001	Remove rows with string "Null" in any column	Customer, Order	Filter: `(col != "Null") & (col.isNotNull())` for string columns
-002	Remove rows with actual NULL in any column	Customer, Order	Filter: `col.isNotNull()` for all columns
-003	Remove exact duplicate rows	Customer, Order	Spark `dropDuplicates()` without column specification
-004	Validate numeric types for PricePerUnit and Qty	Order	Cast to decimal(10,2) and integer; fail job if casting fails
-005	Validate date type for Date column	Order	Cast to date type; fail job if parsing fails
-006	Ensure non-negative TotalAmount in aggregation	Customer Aggregate	Log warning if negative; do not exclude (business decision requi

Deduplication Rules

	Entity	Deduplication Key	Method	Deterministic Selection	
	----------	----------	----------	----------	
	Customer	All columns (CustId, Name, EmailId, Region)	Spark dropDuplicates()	No (first occurrence)	
	Order	All columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)	Spark dropDuplicates()	No (first occurrence)	

• Note: For deterministic duplicate selection, add `.orderBy("CustId")` or `.orderBy("OrderId")` before `dropDuplicates()`.

Join Rules

Join ID	Left Table	Right Table	Join Type	Join Key	Null Handling
---------	----------	----------	----------	----------	----------
JOIN-001	df_customer_clean	df_order_clean	Inner	CustId	Customers without orders excluded
JOIN-002	df_customer_clean	df_existing_customers	Left Anti	CustId	Identifies new customers
JOIN-003	df_customer_clean	df_existing_customers	Inner	CustId (with attribute comparison)	Identifies changed customers

Aggregation Rules

	Aggregation ID	Source Tables	Group By Columns	Aggregate Function	Output Column	
	----------	----------	----------	----------	----------	
	AGG-001	df_customer_clean + df_order_clean	Name, Date	SUM(PricePerUnit × Qty)	TotalAmount	

SCD / History Tracking Rules

Rule ID	Table	SCD Type	Record Key	Change Detection Columns	Active Flag	Start Date	End Date	Operation Times
-------	----------	----------	----------	----------	----------	----------	----------	----------
-001	ordersummary	Type 2	CustId, OrderId	Name, EmailId, Region	IsActive	StartDate	EndDate	OpTs

• SCD Type 2 Logic:

1. New Record: Insert with IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()

2. Changed Record:

• Close existing: Update IsActive=false, EndDate=current_date(), OpTs=current_timestamp()

• Insert new: IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()

3. Unchanged Record: No action

• Hudi Configuration for SCD Type 2:

• Record Key: `CustId,OrderId`

• Precombine Key: `OpTs` (latest timestamp wins)

• Table Type: Copy-On-Write (COW)

• Operation: Upsert

Null Handling

	Column Type	Null Handling Strategy	
	----------	----------	
	String	Remove rows where column equals "Null" (string) OR column is NULL	
	Numeric	Remove rows where column is NULL (string "Null" not applicable after type casting)	
	Date	Remove rows where column is NULL (string "Null" not applicable after type casting)	
	Boolean	No null handling (IsActive always set to true or false)	
	Timestamp	No null handling (OpTs always set to current_timestamp())	

Type Casting Expectations

Source Column	Source Type (CSV)	Target Type (Spark)	Casting Method	Error Handling
----------	----------	----------	----------	----------
CustId	string	string	No casting required	N/A
Name	string	string	No casting required	N/A
EmailId	string	string	No casting required	N/A
Region	string	string	No casting required	N/A
OrderId	string	string	No casting required	N/A
ItemName	string	string	No casting required	N/A
PricePerUnit	string	decimal(10,2)	`col.cast("decimal(10,2)")`	Fail job if casting fails
Qty	string	integer	`col.cast("int")`	Fail job if casting fails

Date	string	date	`to_date(col)` or `col.cast("date")`	Fail job if parsing fails
IsActive	N/A	boolean	`lit(True)` or `lit(False)`	N/A (derived column)
StartDate	N/A	date	`current_date()`	N/A (derived column)
EndDate	N/A	date	`lit(None).cast("date")`	N/A (derived column)
OpTs	N/A	timestamp	`current_timestamp()`	N/A (derived column)
TotalAmount	N/A	decimal(20,2)	`(PricePerUnit * Qty).cast("decimal(20,2)")`	Log warning if negative, do not fail

•

6. Traceability Matrix

Source (FRD)	Technical Source (TRD)
----------	----------
-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/
-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/
mer_raw (from TR-INGEST-001)	df_customer_raw
_raw (from TR-INGEST-002)	df_order_raw
mer_null_removed (from TR-CLEAN-001)	df_customer_null_removed
_null_removed (from TR-CLEAN-002)	df_order_null_removed
mer_clean (from TR-CLEAN-003)	df_customer_clean
_clean (from TR-CLEAN-004)	df_order_clean
mer_clean + df_order_clean + existing ordersummary	df_customer_clean + df_order_clean + s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
mer_clean + existing ordersummary	df_customer_clean + s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
mer_clean + df_order_clean	df_customer_clean + df_order_clean

•

7. ETL Column-Level Mapping

TR-INGEST-001: Customer CSV to df_customer_raw

Source Column (CSV)	Source Data Type	Target Column (DataFrame)	Target Data Type	Transformation	Notes
----------	----------	----------	----------	----------	----------
CustId	string	CustId	string	Direct map	Unique customer identifier
Name	string	Name	string	Direct map	Customer name
EmailId	string	EmailId	string	Direct map	Customer email address
Region	string	Region	string	Direct map	Customer geographic region

TR-INGEST-002: Order CSV to df_order_raw

rce Column (CSV)	Source Data Type	Target Column (DataFrame)	Target Data Type	Transformation	Notes
----------	----------	----------	----------	----------	----------

erId	string	OrderId	string	Direct map	Unique order identifier
nName	string	ItemName	string	Direct map	Product/item name
cePerUnit	string	PricePerUnit	decimal(10,2)	Cast to decimal(10,2)	Price per unit of item
	string	Qty	integer	Cast to integer	Quantity ordered
e	string	Date	date	Cast to date	Order date
tId	string	CustId	string	Direct map	Foreign key to custome

TR-SCD2-001: df_customer_clean + df_order_clean to ordersummary

lumn	Source Table	Target Column	Target Data Type	Transformation	Notes
--------	----------------------	-----------------	------------------	--	------------------------------
	df_customer_clean	CustId	string	Direct map	Customer identifier
	df_customer_clean	Name	string	Direct map	Customer name
	df_customer_clean	EmailId	string	Direct map	Customer email
	df_customer_clean	Region	string	Direct map	Customer region
	df_order_clean	OrderId	string	Direct map	Order identifier
	df_order_clean	ItemName	string	Direct map	Item name
nit	df_order_clean	PricePerUnit	decimal(10,2)	Direct map	Price per unit
	df_order_clean	Qty	integer	Direct map	Quantity
	df_order_clean	Date	date	Direct map	Order date
	N/A	IsActive	boolean	Derived: lit(True) for new/updated	SCD Type 2 active flag
	N/A	StartDate	date	Derived: current_date()	SCD Type 2 effective star
	N/A	EndDate	date	Derived: null for active, current_date() for closed	SCD Type 2 effective end
	N/A	OpTs	timestamp	Derived: current_timestamp()	Operation timestamp in U

TR-AGG-001: df_customer_clean + df_order_clean to customeraggregatespend

rce Column	Source Table	Target Column	Target Data Type	Transformation	Notes
----------	----------------------	-----------------	------------------	--	------------------------------
ne	df_customer_clean	Name	string	Direct map (group by)	Customer name
e	df_order_clean	Date	date	Direct map (group by)	Order date
ePerUnit	df_order_clean	TotalAmount	decimal(20,2)	Derived: SUM(PricePerUnit × Qty)	Total spend per customer per dat
	df_order_clean	TotalAmount	decimal(20,2)	Derived: SUM(PricePerUnit × Qty)	Total spend per customer per dat

-

8. Assumptions and Missing Information

The following information was not specified in the FRD and is assumed or requires clarification:

1. Data Types: Exact data types for CustId, Name, EmailId, Region, OrderId, ItemName were not specified. Assumed all are string type.

2. Numeric Precision: PricePerUnit precision (10,2) and TotalAmount precision (20,2) are assumed based on typical financial data requirements.

3. Date Format: Date field format in Order CSV is assumed to be parseable by Spark's default date parser (ISO 8601 or similar).

4. Timezone: OpTs timestamp is assumed to use UTC timezone.

5. Hudi Table Type: Copy-On-Write (COW) is assumed for ordersummary; Merge-On-Read (MOR) could be considered for write-heavy workloads.

6. Partitioning: No partitioning is specified; recommendations provided for future optimization.

7. Change Detection Mechanism: Incremental processing relies on comparing current customer data against existing ordersummary records; no external CDC mechanism is specified.

8. Duplicate Selection: Spark's dropDuplicates() is non-deterministic; deterministic selection requires explicit ordering.

9. Negative TotalAmount Handling: Business rule for handling negative aggregate amounts is not specified; currently logs warning but does not exclude.

10. Schema Evolution: No schema evolution strategy is specified; Glue Schema Registry or Hudi schema evolution features could be added if needed.

11. Job Scheduling: Job execution frequency (daily, hourly, event-driven) is not specified.

12. Data Retention: Retention policy for historical SCD Type 2 records is not specified.

13. Performance Tuning: Specific performance requirements (SLA, throughput) are not specified.

14. Error Notification: Mechanism for alerting on job failures (SNS, email) is not specified.

15. Data Validation Thresholds: Acceptable thresholds for data quality issues (e.g., max % of null records) are not specified.

• Note: Not enough information available in the FRD to derive source/target schema details for the following (all schema details have been provided based on available information and reasonable assumptions):

• All schemas have been documented based on FRD content and standard AWS Glue/Spark data type conventions.

•

• End of Technical Requirements Document